Encode the thought - neural network training for understanding semantic instead of predicting tokens

“Thoughts die the moment they are embodied by words.” A. Schopenhauer

Lyubimkov Dmitri

E-mail: loftlong@gmail.com

2026

Abstract

This paper presents an experimental proof-of-concept for encoding an invariant semantic core of text ("thoughts") instead of character-by-character token prediction. We propose the Encode_thoughtV2 architecture, which combines a Set Transformer for permutation-invariant encoding, a learnable query engine with Cross-Attention for extracting a compact matrix of semantic slots, and a transformer decoder for reconstruction. The pipeline is model-independent and runs on top of frozen base encoders. Experiments were conducted on the TinyStories dataset using prajjwal1/bert-mini in the most minimal configuration (1 slot, 64 internal dimensions, ~0.9 million trainable parameters). Training was performed using a combined loss function (CrossEntropy + CosineLoss) in parallel Teacher Forcing mode using Context Dropout. The results show that the architecture successfully extracts meaning and recovers 80–90% of the source text in the presence of ideal context (Corrected mode), empirically confirming the hypothesis of low effective dimensionality of the semantic core of simple narratives. However, closed-loop autoregression (AR and Raw AR) collapses into repetitive patterns due to exposure bias, not due to capacity constraints or architectural flaws. Parameter scaling does not eliminate this gap. The study concludes the successful completion of the first phase: slot-based compression and reconstruction have proven their effectiveness. Further research will focus exclusively on stabilizing the autoregressive loop through a shift in training paradigm, as without robust symbol-by-symbol recovery, the practical value of the concept remains limited.

Table of contents

Abstract....1
1. Encode thought Version 1....3
2. Goals and objectives....3
3. Selection of basic transformer encoders....3
4. Selecting a neural network architecture....4
4.1. Statement of the problem and requirements for presentation....4
4.2. Analysis of alternative approaches....4
4.3. Selected architecture....5
4.3.1. Encoder: Set Transformer with ISAB blocks....5
4.3.2. Slot Mechanism: Learnable Queries + Cross-Attention....5
4.3.3. Decoder: Transformer Decoder....5
5. Selecting a dataset....6
6. Preparing for launch....6
6.1. Launch Platform....6
6.2. Loading a project....6
6.3. Preparing the virtual environment....6
6.4. Project files and directories....7
7. Preparing the dataset....8
8. Models training....8
9. Analysis of results....8
10. Encode_thoughtV2 model parameters....9
10.1. 10.1 Model Independence and Unified Pipeline....9
10.2. Final Configuration....9
10.3. Loss Function and Learning Strategy....9
10.4. Key engineering decisions....9
10.5. Limits of applicability and diagnostics....10
11. Results of the analysis of the Encode_thoughtV2 model....10
11.1. Results for the prajjwal1/bert-mini encoder....10
11.2. Analysis of results....11
11.2.1. Corrected mode (Teacher Forcing)....12
11.2.2. AR and Raw AR modes....12
11.2.3. Conclusions....12
12. Direction for continuing research and experiments....12

1. Encode thought Version 1

The first version of Encode thought naively considered the idea and necessity of encoding the semantics/meaning of text instead of words/tokens. It drew an analogy and distinction between words and thoughts and raised the issue of the lack of explicit thought identification in modern LLMs. As an alternative, it proposed encoding not individual words/tokens but semantically complete constructs, such as sentences. A possible mechanism for such encoding was the use of a compact bottleneck encoder-decoder pair.

Version 1 remained theoretical—it articulated the idea but lacked any architectural choices or experimental verification. This paper (Encode_thoughtV2) aims to fill this gap. The full text of Version 1 can be found at https://github.com/loftyara/Encode_thought/ or https://loftyara.github.io/encode_thought.html

Version 1 remained theoretical—it articulated the idea but lacked any architectural choices or experimental verification. This paper (Encode_thoughtV2) aims to fill this gap. The full text of Version 1 can be found at https://github.com/loftyara/Encode_thought/ or https://loftyara.github.io/encode_thought.html

2. Goals and objectives

This paper aims to experimentally validate the concept proposed in Encode_thought (Version 1). While the first version remained theoretical, Encode_thoughtV2 implements it in practice through model training and source text recovery experiments.

The main objectives of the project:

- Selection of basic LLM models for encoding and restoring tokens
- Selecting a neural network architecture for text semantic extraction
- Selection and compilation of datasets for testing hypotheses
- Training a neural network for semantic encoding and source text recovery
- Recovering source texts and measuring the parameters of correspondence between source and recovered data
- • Hypothesis testing

3. Selection of basic transformer encoders

Regardless of how we attempt to extract semantics, we still need to translate text to embeddings and vice versa. This requires encoders, but these encoders don't require significant knowledge and, therefore, a significant size, as we expect our target neural network to handle this. This encoder must understand the text, its syntax, and be able to transform a sequence of tokens into a sequence of embeddings.

We could train such an encoder ourselves during our experiments, but we don't yet have any specific requirements for such an encoder. Therefore, we'll use a ready-made model. Decoder-only (GPT-like) models aren't suitable because we don't need to predict the next token; we need to obtain a vector representation (embedding) of the current one. Also, for experiments on simple short texts, we don't need large models. Table 1 lists basic transform encoders.

#	Model	Hidden dimension	Layers / Options	Training strategy
1	distilbert-base-uncased	768	6 / ~66M	Knowledge Distillation + MLM
2	prajjwal1/bert-mini	256	4 / ~11M	Pure MLM (compressed architecture)
3	all-MiniLM-L6-v2	384	6 / ~22.7M	Distillation + Contrastive Learning
4	TinyBERT_General_4L_312D	312	4 / ~14.2M	Multi-layer Distillation + MLM
5	jina-embeddings-v2-small-en	512	~8 / ~33M	Contrastive Learning (Sentence-level)

Table 1. Basic transformer encoders

We'll try each model from this list to further understand how the obtained results depend on the base encoder. All models were initialized with weights from the Hugging Face Hub and used without additional training during the embedding generation stage.

4. Selecting a neural network architecture

A child, beginning to understand speech, is able to immediately extract meaning from it. Perhaps this means that semantic extraction is not a complex task and should not require a complex neural network architecture. Even blind children can understand speech, which perhaps means that speech/text already contains everything necessary for semantic extraction and does not require additional grounding. Based on these principles, we will attempt to select a neural network architecture from existing ones—one that is sufficiently simple and focused on text processing.

4.1. Statement of the problem and requirements for presentation

The goal of the neural network is to extract an invariant semantic core from the text—a representation that preserves meaning but abstracts from form. We assumed that syntax and specific language are merely encoding protocols, while meaning exists independently of them. Based on theoretical analysis and practical constraints, the following requirements for the target representation were formulated:

1. Vector matrix. A single vector of fixed dimensions creates a bottleneck, inevitably leading to the loss of detail and connections in texts of varying complexity. Meaning should be encoded as a set of vectors (slots), where each slot represents a distinct aspect of the information.
2. Implicit structure. Structural relationships (entities, actions, connections) should emerge naturally during the learning process through the geometry of the latent space, rather than being imposed through explicit graph diagrams or discrete rules.
3. Language and syntax invariance. Architectural invariance to token order (Set Transformer) and potential cross-lingual stability achieved through syntax abstraction in slots.
4. Training stability. The architecture must be mathematically sound and predictable. The model must converge reliably without fragile mechanisms sensitive to initialization or hyperparameters.
5. End-to-end learning. Semantics should be extracted directly from a sequence of tokens, without intermediate symbolic transformations.

4.2. Analysis of alternative approaches

During the design process, the following architecture classes from Table 2 were considered and rejected:

#	Approach	Reason for refusal
1	Single Vector Embeddings (LaBSE, sentence-transformers)	Insufficient capacity to fully encode meaning. Suitable for retrieval and classification, but unsuitable for tasks requiring the preservation of detail and structure.
2	Slot Attention	It uses iterative slot competition via softmax, which creates positive feedback and leads to training instability, sensitivity to initialization, and the risk of slot collapse. It violates the principle of robustness.
3	Capsule Networks	Conceptually similar to the idea of structured vectors, they exhibit practical problems with training and scaling on text data. The lack of proven results in NLP makes this approach risky.
4	VQ-VAE	Focuses on discrete compression, which contradicts the requirement for a continuous semantic space with a geometric structure. It does not provide an explicit decomposition of meaning into aspects.

5	Explicit graph representations (AMR, SRL)	They require complex parsers, introduce discrete constraints, and don't adhere to the principle of learning from streams. The graph is an artifact of analysis, not a natural form of meaning.

Table 2. Rejected architecture classes

4.3. Selected architecture

The resulting architecture is a hybrid of Set Transformer and the Learnable Queries mechanism, combining the mathematical rigor of working with sets with the robustness of transformer blocks:

Text → Token Embeddings → Set Transformer Encoder → Hidden States

↓

Learnable Queries → Cross-Attention → Slot Matrix (N × D)

↓

Transformer Decoder → Text reconstruction

4.3.1. Encoder: Set Transformer with ISAB blocks

The encoder used is a Set Transformer, specifically designed for processing data in the form of sets. Key features:

- Order invariance. The architecture is constructed independently of permutations of input elements, eliminating the need to memorize position-specific syntactic patterns.
- Efficiency. Using Induced Set Attention Blocks (ISAB) provides linear complexity O(n) in the sequence length, which is critical for scaling.
- Stability. The attention mechanisms in Set Transformer are mathematically determined and do not contain iterative or competitive processes.

4.3.2. Slot Mechanism: Learnable Queries + Cross-Attention

Extraction of semantic slots is implemented through a set of trainable query vectors that independently aggregate information from the encoder outputs:

- No contention. Each query interacts with the encoder via standard cross-attention without any mechanisms to suppress other slots. This ensures smooth gradients and stable convergence.
- Natural specialization. The division of responsibility between slots arises under the pressure of the reconstruction loss function: duplicating information is disadvantageous, as it increases the error. Slots independently distribute aspects of meaning (entities, relations, details) during the optimization process.
- Flexibility. The number of slots and their dimensions are hyperparameters that allow the representation capacity to be adapted to the complexity of the problem.

4.3.3. Decoder: Transformer Decoder

The decoder accepts the slot matrix and reconstructs the embedding sequence of the base model. The subsequent transition to tokens occurs during the analysis phase. Using a standard transformer decoder ensures:

- Compatibility with modern generation practices.
- Possibility of cross-lingual recovery (encoding in one language, decoding in another), which clearly forces the model to highlight the invariant meaning.

The selected architecture satisfies all the formulated requirements:

#	Requirement	Implementation
1	Matrix of vectors	A set of N slots of dimension D
2	Implicit structure	Relationships are encoded by slot geometry, specialization occurs through Loss
3	Invariance	Set Transformer (order invariance) + multilingual (language invariance)
4	Stability	No competition or iteration, standard transformer blocks
5	End-to-end	Direct learning from a sequence of embeddings without intermediate discrete representations.

Table 3. Compliance of the selected architecture with the requirements

5. Selecting a dataset

The TinyStories dataset was chosen to experimentally test the hypothesis regarding the feasibility of extracting an invariant semantic core from text. The project's goal is to test whether a compact architecture can extract meaning from text without relying on encyclopedic knowledge or complex linguistic constructs. TinyStories is ideal for this purpose: the dataset consists of short stories (50–300 words) written in simple language at the preschool level. The stories contain a clear cause-and-effect structure, explicit entities and actions, but minimize syntactic complexity and ambiguity. This allows us to isolate the task of semantic compression from the tasks of world understanding or lexical ambiguity resolution. The dataset consists of two text files, train and val, with stories ending with the <|endoftext|> tag.

6. Preparing for launch

6.1. Launch Platform

All experiments were conducted on the author's PC with the following characteristics:

- OC Windows 11
- 32 GB DRAM
- Nvidia 5070 Ti graphics card with 16 GB of memory (CUDA support)
- Python version 3.12

6.2. Loading a project

The Encode_thoughtV2 project is hosted in an open repository on GitHub. To get it to your computer, follow these steps:

6.2.1. Open the command prompt (cmd)

6.2.2. Navigate to the directory where you want to place the project (for example, D:\Projects with the command cd D:\Projects)

6.2.3. Run the command git clone https://github.com/loftyara/Encode_thoughtV2.git

6.2.4. Instead of the command line, you can use the GitHub Desktop program

6.2.5. Once complete, you'll see a new folder called Encode_thoughtV2. The entire project structure will be contained within it.

6.3. Preparing the virtual environment

Для изоляции зависимостей проекта рекомендуется использовать виртуальное окружение Python (venv). Перейдите в каталог загруженного проекта и запустите команду:

```
python -m venv venv
```

Activate the virtual environment:

venv\Scripts\activate

Install pytorch:

pip install torch --index-url https://download.pytorch.org/whl/cu130

Install dependencies:

pip install -r requirements.txt

6.4. Project files and directories

The list of directories is given in Table 4, the list of files in Table 5.

#	Directory	Description
1	checkpoints	saving models and/or their checkpoints
2	data	Dataset
3	data/raw	raw data of the dataset
4	data/processed	calculated embeddings
5	docs	documentation
4	scripts	data preparation programs
7	src	programs for training models and analyzing results

Table 4. List of project directories

#	File	Directory	Description
1	best_bert-mini.pt	checkpoints	A model based on prajjwal1/bert-mini. It's being created during training.
2	train.txt	data/raw	Dataset, training portion. Loaded during runtime.
3	val.txt	data/raw	Dataset, validation part. Loaded during runtime.
4	embeddings_all-MiniLM-L6-v2_train_chunk_nnnn.pt	data/ processed	All-MiniLM-L6-v2 embeddings. Calculated during runtime.
5	embeddings_bert-mini_train_chunk_nnnn.pt	data/ processed	prajjwal1/bert-mini embeddings. Calculated during runtime.
6	embeddings_distilbert-base-uncased_train_chunk_nnnn.pt	data/ processed	distilbert-base-uncased embeddings. Calculated during runtime.
7	embeddings_jina-embeddings-v2-small-en_train_chunk_nnnn.pt	data/ processed	jina-embeddings-v2-small-en embeddings. Calculated during runtime.
8	embeddings_TinyBERT_General_4L_312D_train_chunk_nnnn.pt	data/ processed	TinyBERT_General_4L_312D embeddings. Calculated during runtime.
9	encode_thought.pdf	docs	documentation
10	01_download_dataset.py	scripts	dataset loading program
11	02_gen_embeddings_bertmini.py	scripts	embedding calculation program all-MiniLM-L6-v2.
12	02_gen_embeddings_distilbert.py	scripts	embedding calculation program prajjwal1/bert-mini.
13	02_gen_embeddings_jina.py	scripts	embedding calculation program distilbert-base-uncased.
14	02_gen_embeddings_minilm.py	scripts	embedding calculation program jina-embeddings-v2-small-en.
15	02_gen_embeddings_tinybert.py	scripts	embedding calculation program TinyBERT_General_4L_312D.
16	dataset.py	src	library for accessing the dataset (embeddings)
17	model.py	src	library for training the semantic model
18	01_train_bertmini.py	src	model training program for prajjwal1/bert-mini embeddings.

19	02_analyze_slots_bertmini.py	src	анализ модели для эмбидингов prajjwal1/bert-mini embeddings.

Table 5. List of project files

7. Preparing the dataset

Activate the virtual environment:

venv\Scripts\activate

Go to the scripts directory:

cd scripts

Run the program:

python 01_download_dataset.py

Run the programs one by one:

python 02_gen_embeddings_bertmini.py

python 02_gen_embeddings_distilbert.py

python 02_gen_embeddings_jina.py

python 02_gen_embeddings_minilm.py

python 02_gen_embeddings_tinybert.py

8. Models training

Activate the virtual environment:

venv\Scripts\activate

Go to the src directory:

cd src

Run the programs one by one:

python 01_train_bertmini.py

The author was unable to achieve his goals, so the experiments have so far been limited to the basic prajjwal1/bert-mini transform encoder.

9. Analysis of results

Activate the virtual environment:

venv\Scripts\activate

Go to the src directory:

cd src

Run the program for the model whose training results you want to see:

python 02_analyze_slots_bertmini.py

The author was unable to achieve his goals, so the experiments have so far been limited to the basic prajjwal1/bert-mini transform encoder.

10. Encode_thoughtV2 model parameters

10.1. 10.1 Model Independence and Unified Pipeline

The EncodeThoughtModel architecture is intentionally designed as a model-independent module. It does not contain a rigid binding to the dimensionality, number of layers, or internal structure of the underlying encoder. The input tensor (B, T, d_in) first passes through a linear projection input_proj, which adapts the arbitrary dimensionality of the underlying LLM embeddings (d_in ∈ [256, 768]) to a fixed internal model space (dim_model). The decoder, in turn, dynamically shapes the output layer output_proj to the dimensionality d_in, ensuring an exact correspondence with the original latent space.

10.2. Final Configuration

The final parameters are given in Table 6.

#	Parameter	Value	Role
1	NUM_SLOTS	8	Number of trainable slot vectors (semantic representation capacity)
2	DIM_MODEL	64	Internal dimensions of transformer blocks
3	NUM_HEADS	1	Attention heads (sufficient for dim_model=64, eliminates gradient sparsification)
4	NUM_ENCODER_LAYERS	1	ISAB layers in the Set encoder
5	NUM_DECODER_LAYERS	1	Cross-Attention Layers in the Decoder
6	NUM_INDS	4	Induced elements to reduce attention complexity to O(T)
7	MAX_SEQ_LEN	51	Hardware limit of decoder positional embeddings

Table 6. Final parameters of the model

The total number of trainable parameters is ~0.8–0.9 million (depending on the input_dim of the base LLM). For comparison, base encoders contain between 11 and 66 million parameters. The model performs compression, invariant extraction, and reconstruction using less than 2% of the computational effort of the original architecture.

10.3. Loss Function and Learning Strategy

Instead of the classical MSE, a combined loss function is used: $L = \alpha$ CrossEntropy + β CosineLoss

- CrossEntropy (with label_smoothing=0.1) ensures accurate targeting of the token and prevents the formation of over-peaked distributions, which lead to cycling in autoregression.
- CosineLoss maintains the geometric proximity of the predicted embeddings to the original space of the base model, acting as a soft regularizer and keeping vectors within the working manifold.

Training is performed in Parallel Teacher Forcing mode. To improve the decoder's resilience to distribution shifts, Context Dropout (CONTEXT_DROP_PROB=0.15) is used: with a specified probability, the entire input context is replaced with a BOS token, forcing the model to learn to generate a sequence based solely on slots. This eliminates the need for heavy dropout layers and does not disrupt parallelism.

10.4. Key engineering decisions

- Full parallel training: No stepping loops or retain_graph. An epoch on the minimum configuration takes 3-5 seconds, VRAM consumption is strictly constant and does not depend on the sequence length.
- Tied Embeddings: The decoder output layer uses the weight matrix of the base encoder dictionary. This ensures the consistency of the latent space and eliminates mismatches when searching for nearby tokens.

- Loss masking: Padding tokens are excluded from the loss calculation. Gradients are formed only at real positions, which prevents signal blurring and accelerates convergence.
- Deterministic initialization: xavier_uniform_ for slots, induced elements, and the BOS token ensures a stable start and reproducible results on any GPU.

10.5. Limits of applicability and diagnostics

The experiments clearly divided the capabilities of the architecture into two modes:

1. 1. Corrected (Teacher Forcing): The model recovers 80–90% of the semantic content even with NUM_SLOTS=1 and DIM_MODEL=64. This confirms that the proposed architecture successfully extracts an invariant semantic core and is capable of accurate reconstruction in the presence of an ideal context.
2. 2. AR / Raw AR: In closed-loop autoregression, generation collapses into repeating patterns. Diagnostics revealed that the cause is not a lack of capacity or architectural errors, but rather exposure bias: the model is trained on an ideal context distribution and does not detect cascading drift in its own predictions. Increasing the model size or the number of epochs does not eliminate this gap.

Conclusion: The current stack fully solves the problem of compact semantic compression and reconstruction in Teacher Forcing mode. Robust autoregression requires a change in training paradigm (sequence-level optimization, online truncated BPTT, or RL) and is a separate research stage. The architecture is fixed as a working, scalable module for semantic representation, search, and cross-lingual transfer tasks.

11. Results of the analysis of the Encode_thoughtV2 model

11.1. Results for the prajjwal1/bert-mini encoder

The results are shown in Table 7.

#	Original text	Recovered text
1	Spot. Spot saw the shiny car and said, "Wow, Kitty, your car is so bright and clean!" Kitty smiled and replied, "Thank you, Spot. I polish it every day." After playing with the car, Kitty and Spot felt thirsty. They found a small pond with clear water. They drank the water and felt very happy. They played together all day and became best friends.	spot. spot saw the shiny car and said, " wow, kitty, your car is so bright and clean! " kitty smiled and replied, " thank you, spot. i blew it every day. " after playing with the car, kitty and spot felt hungry. they found a small pond with clear water. they drank the water and felt very happy. they played together all day and became best best best
2	Once upon a time, in a big forest, there lived a rhinoceros named Roxy. Roxy loved to climb. She climbed trees, rocks, and hills. One day, Roxy found an icy hill. She had never seen anything like it before. It was shiny and cold, and she wanted to climb it. Roxy tried to climb the icy hill, but it was very slippery. She tried again and again, but she kept falling down. Roxy was sad. She wanted to climb the icy hill so much. Then, she saw a little bird named Billy. Billy saw that Roxy was sad and asked, "Why are you sad, Roxy?" Roxy told Billy about the icy hill and how she couldn't climb it. Billy said, "I have an idea! Let's find some big leaves to put under your feet. They will help you climb the icy hill." Roxy and Billy looked for big leaves and found some. Roxy put the leaves under her feet and tried to climb the icy hill again. This time, Roxy didn't slip. She climbed and climbed until she reached the	once upon a time, in a big forest,, lived a chesterceros named roxy.. wanted to climb. she climbed trees, rocks, and hills. one day,, found an icy hill. she had never seen anything like it before. it was shiny and cold, and she wanted to climb it.. tried to climb the icy hill, but it was very dangerous. she tried again and again, but but kept falling down. roxy was sad. she wanted to climb the the hill so much. then, she saw a little bird named billy. billy saw that roxy was sad and asked, " why are you sad, sam? " roxy told billy about the icy hill and who she couldn't climb it. billy said said " i have an idea! let's find some big leaves to put under your feet. they will help you climb the icy hill. " sam and billy looked for big leaves and found some. roxy put the leaves under her feet and tried to climb the icy hill again again. time,, didn't slip. she climbed and climbed

	top of the icy hill. Roxy was so happy! She and Billy played on the icy hill all day. From that day on, Roxy and Billy were the best of friends, and they climbed and played together all the time. And Roxy learned that with a little help from a friend, she could climb anything.	until she reached the top of the icy hill.. was so happy! she and billy played on the icy hill that day. from that day on,, and billy were the best of friends, and they climbed and played together all the time. and and learned that with a little help from a friend, she could climb climb climb
3	Once upon a time, in a small yard, there was a small daisy. The daisy had a name. Her name was Daisy. Daisy was very small, but she was also very happy. One day, Daisy saw a dog. The dog was big and had a name too. His name was Max. Max liked to play in the yard. Daisy liked to watch Max play. Max and Daisy became friends. Every day, Max would come to the yard to play. Daisy would watch and smile. They were very happy together. And even though Daisy was small, she knew that she had a big friend in Max.	once upon a time, in a small yard, there was a small daisy. the daisy had a name. her name was daisy. daisy was very small, but she was also very happy. one day, daisy saw a dog. the dog was big and had a name too. his name was max. max liked to play in the yard. daisy liked to watch max play. max and daisy became friends. every day, max would come to the yard to play. daisy would watch and smile. they were very happy together. and even though daisy was small, she knew that she had a big friend in in in
4	Once upon a time, there was a thoughtful girl named Sue. Sue loved to help her mom around the house. One day, her mom asked her to wipe the table after they ate their lunch. Sue was happy to help. As Sue was wiping the table, she saw a pretty candle on the window sill. The candle was her mom's favorite. Sue wanted to do something nice for her mom, so she said, "Mom, can I light the candle for you?" Her mom said, "Yes, but be very careful." Sue carefully lit the candle and put it on the table. Her mom was so happy to see the pretty candle. They both sat and watched the candle burn. Sue's mom said, "Thank you, Sue, for being so thoughtful and careful." Sue felt proud that she could help her mom. The moral of the story is to always be thoughtful and careful when helping others.	once upon a time, there was a thoughtful girl named sue. sue wanted to to her mom around the house. one day, her mom asked her to wipe the table after they ate their dinner. sue was happy to help. as sue was wiping the table, she saw a pretty candle on the the sii. the candle was her mom's favorite. sue wanted to do something nice for her mom, so she said, " mom, can i light the candle for you? " her mom said, " yes, but be very careful. " sue carefully lit the candle and put it on the table. her mom was so happy happy see the pretty candle. they both sat and watched the candle burn..'s mom said, " thank you, sue, for being so thoughtful and careful. " sue felt proud that she could help her mom. the moral of the story is is always be thoughtful and careful when helping helping helping
5	Once upon a time, there was a kind farmer. He had a big cow. The cow was sad. The farmer did not know why. One day, a little boy came to the farm. He saw the sad cow. The boy kneeled down to talk to the cow. "Why are you sad, cow?" he asked. The cow said, "I am lonely. I want a friend." The kind farmer heard the cow. He wanted to help. So, he got another cow to be friends with the sad cow. The sad cow was happy now. They played together every day. And the kind farmer, the little boy, and the two cows all lived happily ever after.	once upon a time, there was a kind farmer. he had a big cow. the the was sad. the farmer did not know why. one day, a little boy came to the farm. he saw the sad cow. the boy kneeled down to talk to the the. " why are you sad, cow? " he asked. the cow said, " i am lonely. i want a friend. " the kind farmer heard the the. he wanted to help. so, he got another cow to to friends with the sad cow. the sad cow was happy now. they played together every day. and the kind farmer, the little boy, and the two children all lived happily ever ever ever

Table 7. Results for the prajjwal1/bert-mini encoder

11.2. Analysis of results

All main experiments were conducted with the prajjwal1/bert-mini baseline encoder (latent dimension 256, ~11 million parameters, pretraining strategy: pure MLM). To test the hypothesis of low effective semantic core dimensionality, the model's minimal configuration was used: NUM_SLOTS=1, DIM_MODEL=64, one layer each in the encoder and decoder, NUM_INDS=4. The final number of trainable parameters was less than 1 million.Тестировалось восстановления текста в трёх режимах: Corrected (Teacher Forcing с идеальным контекстом), AR (авторегрессия с квантованием токенов) и Raw AR (авторегрессия на сырых эмбеддингах).

11.2.1. Corrected mode (Teacher Forcing)

Even with the minimum configuration, the model demonstrates reliable reconstruction of the source text. The storyline, entity names, cause-and-effect relationships, and overall style are preserved. Lexical accuracy reaches 80–90%.

The observed deviations are predictable:

• Semantically close replacements (polish → blew, difficult → rough) that do not distort the meaning.

• BERT subtoken artifacts ([unused...], yffb) caused by misses on adjacent dictionary elements due to aggressive compression.

• Local repetitions of final words (worked worked, ever ever) associated with high model confidence at sentence boundaries.

This result empirically confirms that the architecture successfully extracts an invariant semantic core and is capable of accurate reconstruction when the decoder relies on an ideal context.

11.2.2. AR and Raw AR modes

When switching to closed-loop autoregression, generation fails altogether. After just 5-10 steps, the model devolves into rigid cyclical repetitions of high-frequency tokens (and, was, the, popular) or meaningless patterns. Raw AR collapses similarly, sometimes adding a drift toward abstract or technical words.

Diagnostics clearly showed that the problem was not a coding error, a slot architecture issue, or a lack of parameters. The model was trained exclusively on an ideal context distribution (Exposure Bias). At the first deviation from the ideal, the decoder's self-attention defocuses, and the system falls into the nearest lexical attractor, with no mechanism for returning to meaning via slots. Increasing the model size, the number of epochs, or adding regularization does not eliminate this gap.

11.2.3. Conclusions

The current version of the architecture fully solves the problem of compact semantic compression and reconstruction in Teacher Forcing mode. Autoregressive generation is currently ineffective. This isn't a failure of the concept, but rather a clear limitation of applicability: slot-based compression and context-aware reconstruction have proven their effectiveness even on an extremely small model. Robust AR generation requires a shift in the training paradigm (sequence-level optimization, online truncated BPTT, or RL) and is a separate research project.

12. Direction for continuing research and experiments

The current results show a clear boundary: the architecture successfully compresses meaning and restores text in the Teacher Forcing (Corrected) mode, but completely loses stability in closed-loop autoregression (AR and Raw AR).

We proceed from a simple principle: without stable, character-by-character reconstruction of the original text, the practical value of the entire concept is limited. Therefore, all related research projects are put on hold. The sole goal of the next stage is to achieve stable AR generation.

What is planned to be done:

1. Inference tests. Check whether the repetition loop breaks at the decoding level: repetition_penalty, adaptive temperature, top_p/top_k, context window limit.
2. Contour stabilization. Implement normalization of raw embeddings before feedback and periodic "anchoring" of the decoder to slots to dampen distribution drift.

3. Learning on your own trajectories. Shift from teacher forcing to closed-loop learning: Truncated BPTT with a fixed window, online scheduled sampling, or sequence-level optimization.

We don't know which of these approaches will work or whether any of them will lead to stable recovery. This is an open experimental task. Increasing the model size, the number of epochs, or changing the dataset are not priorities until the basic "slots → text" loop is closed. The next version of this paper will contain exclusively the results of attempts to stabilize the autoregression.

To be continued …